CHAPTER 8

Caching: CDN, Redis, Memcached

Part II · Building Blocks — the single highest-impact way to make a system fast.

LEARNING OBJECTIVES

- Explain what caching is, why it works, and what the cache hit ratio measures.
- Name the layers where caches live, from the browser to the database.
- Explain what a CDN does and why it slashes latency, origin load, and bandwidth cost.
- Compare Redis and Memcached and choose between them.
- Apply the main caching strategies (cache-aside, write-through, write-back, write-around).
- Handle the hard parts: invalidation, eviction, and failure modes like the thundering herd.

REAL-WORLD ANALOGY — THE LIBRARIAN'S FRONT CART

A librarian notices that a handful of books get requested constantly. Instead of walking deep into the stacks every time, she keeps those few on a cart right at the front desk. Most requests are now answered in seconds; only the rare, unusual request means a trip to the back. A **cache** is that front cart: a small, fast, close-by copy of the data people ask for most.

8.1 The problem caching solves

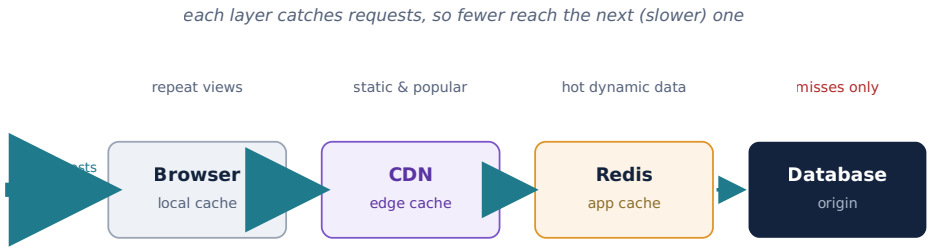
Chapter 2 taught that fetching data down the memory hierarchy is 100–1,000,000× slower, and Chapter 5 showed most real workloads are heavily **read-dominated** — the same data requested over and over. Recomputing a result or re-fetching a row from disk (or across the world) for every identical request is enormous waste. **Caching** stores a copy of frequently-used data somewhere fast and close, so most requests never pay the full cost.

8.2 Why caching works

Caching relies on **locality**: in almost every system, a small fraction of the data accounts for a large fraction of requests (the 80/20 rule; more formally a Zipf distribution). Cache that hot fraction and you serve most traffic from memory. The key metric is the **cache hit ratio** — the percentage of requests answered by the cache. A **hit** is fast; a **miss** falls through to the slower source. Even a 90% hit ratio means the slow path runs one-tenth as often.

8.3 Where caches live — the layers

Caching isn't one thing in one place; it's a series of layers along the request path. Each layer catches some requests so fewer reach the next, slower layer.



The arrow thins at each stage: fewer and fewer requests reach the expensive origin.

8.4 The CDN — caching at the edge

A **Content Delivery Network** caches content on servers spread around the world, so users are served from a nearby **edge** location instead of your distant origin. Recall from Chapters 4–5: a cross-world round trip costs ~150 ms, and image/video egress can be gigabytes per second. A CDN attacks all of that at once — cutting **latency** (content is physically closer), **origin load** (the origin barely gets touched), and **bandwidth cost**. CDNs use **pull** (fetch from origin on first request, then cache) or **push** (you upload content to the CDN ahead of time), and honor `Cache-Control` / TTL headers to decide how long to keep each item.

8.5 Redis vs Memcached

For caching your own dynamic data (not static files), the two classic in-memory stores are **Memcached** and **Redis**. Both keep data in RAM for microsecond access.

	Memcached	Redis
Data types	Simple key–value (strings)	Rich: strings, lists, sets, sorted sets, hashes, streams
Threading	Multi-threaded — simple and fast for pure caching	Mainly single-threaded command execution (very fast per core)
Persistence	None (pure cache)	Optional (snapshotting / append-only log)
Extras	Just caching	Pub/sub, atomic ops, TTLs, Lua scripting, can act as more than a cache
Choose when	You want the simplest possible cache for small values	You need data structures, atomic operations, or richer features

NOTE

Redis is the more common default today because its data structures and atomic operations are broadly useful (recall the atomic rate-limiter counter idea from the resilience patterns). Its licensing changed in 2024, prompting an open-source fork called **Valkey**; for design purposes they behave the same. Pick Memcached only when you truly need nothing beyond a plain key–value cache.

8.6 Caching strategies

“Use a cache” isn’t enough — you must decide *how* reads and writes interact with it.

Strategy	How it works	Trade-off
Cache-aside (lazy)	App checks cache; on miss, reads DB and populates cache. The default.	Simple; first request is a miss; risk of stale data until TTL.
Read-through	App asks the cache, which loads from the DB itself on a miss.	Cleaner app code; needs cache that supports it.
Write-through	Writes go to cache *and* DB together, synchronously.	Cache always fresh; writes are slower.
Write-back (write-behind)	Write to cache now, flush to DB later, in batches.	Very fast writes; risk of data loss if cache dies before flush.
Write-around	Writes go straight to DB, skipping the cache.	Avoids caching write-once data; reads of new data miss first.

Cache-aside is the most common: the application controls the cache explicitly, which is flexible and easy to reason about.

8.7 The hard part: invalidation and eviction

There is an old joke that the two hardest problems in computing are cache invalidation and naming things. Once you cache data, you must decide when the copy is **stale** and must be refreshed or removed.

- **TTL (time to live):** each entry expires after a set time. Simple and self-healing, but data can be stale for up to the TTL.
- **Explicit invalidation:** when the underlying data changes, delete or update the cached entry. Fresher, but you must remember to do it everywhere the data is written.

And when the cache fills up, an **eviction policy** decides what to drop: **LRU** (least recently used — the usual choice), **LFU** (least frequently used), or FIFO. The goal is to keep the hot data and shed the cold.

8.8 Cache failure modes to know

THUNDERING HERD / CACHE STAMPEDE

A very popular key expires, and suddenly *thousands* of requests miss at once and all slam the database together — which can crash it. Fixes: a **lock** so only one request recomputes while others wait; **request coalescing** (merge duplicate concurrent misses into one DB call); or **early/probabilistic expiration** so entries refresh before they all expire simultaneously.

CACHE PENETRATION & HOT KEYS

Penetration: requests for keys that don’t exist always miss and hammer the DB — fix by caching the “not found” result, or using a **Bloom filter** (Chapter 39) to reject unknown keys cheaply. **Hot key:** one key gets so much traffic it overloads a single cache node — fix by

replicating that key across nodes or caching it in the app itself. And beware the **cold start**: after a cache restart everything misses, so “warm” the cache with hot data before taking full traffic.

8.9 When NOT to cache

CACHING IS NOT FREE

A cache adds a moving part, memory cost, and the ever-present risk of serving stale data. Don’t cache data that changes on nearly every request (low reuse → low hit ratio), and be very careful caching data that must be **strongly consistent** and correct to the instant — like an account balance in a payment system. For such data, either don’t cache, or use write-through with tight invalidation. Caching trades a little consistency for a lot of speed; make that trade only where it’s safe.

Common mistakes

WATCH OUT

- Caching without an **invalidation plan**, then serving stale data forever.
- No protection against the **thundering herd** when a hot key expires.
- Not caching **negative results**, letting missing-key requests hammer the DB.
- Caching strongly-consistent data (balances, inventory counts) and showing wrong values.
- Ignoring the **hit ratio** — a cache with a low hit ratio adds cost for little gain.
- Forgetting **cold start** / warming after a cache restart or deploy.

Interview questions

1. What is caching and why does it work? What does the hit ratio measure?
2. Name the caching layers along a request’s path from browser to database.
3. Compare cache-aside and write-through. When would you use each?
4. Compare Redis and Memcached. When would you pick each?
5. What is a thundering herd / cache stampede, and how do you prevent it?
6. How do you handle cache invalidation? Compare TTL vs explicit invalidation.
7. When should you *not* cache something?

Hands-on exercises

1. For 100 requests with an 85% hit ratio, how many reach the database?
2. Decide a caching strategy for: a user’s profile (rarely changes), a live stock price (changes constantly), a shopping-cart total.
3. Design a cache key and TTL for “top 10 posts of the day.”
4. Sketch how you’d stop a stampede when the “home feed” cache entry expires.
5. List which items on a news homepage should come from a CDN vs an app cache vs the DB.

Mini project

ADD A CACHE AND MEASURE THE WIN

Take a small app with a slow endpoint (e.g. one that runs a heavy DB query). Add **Redis** using the **cache-aside** pattern: on each request, check Redis first; on a miss, run the query, store the result with a TTL, and return it. Measure the endpoint's latency and the **hit ratio** before and after warming up. Then delete the underlying data and observe the stale window until the TTL expires — invalidation made concrete.

Large project (capstone continues)

YOUR SOCIAL PLATFORM — THE CACHING STRATEGY

Using your read-heavy Chapter 5 numbers, add caching to your reference architecture. Decide **what** to cache (user profiles, home feeds, post counts), the **strategy** (cache-aside for most), the **TTLs**, and the **invalidation** rules on writes. Put media behind a **CDN**. Identify your likely **hot keys** (a celebrity's profile) and your stampede risk (a viral post's feed entry), and note a mitigation for each. This caching layer is what will let your read QPS be served without melting the database in Part III.

Chapter summary

- **Caching** keeps a fast, close copy of frequently-used data; it works because of **locality** (a small hot fraction serves most requests). The **hit ratio** is the key metric.
- Caches form **layers**: browser → CDN → app/Redis → database; each catches requests so fewer reach the slow origin.
- A **CDN** caches at the edge, cutting latency, origin load, and bandwidth cost for static and popular content.
- **Redis** (rich data structures, optional persistence) vs **Memcached** (simple, pure cache) — pick by whether you need more than plain key–value.
- Choose a **strategy** (cache-aside is the default) and plan **invalidation** (TTL and/or explicit) and **eviction** (usually LRU).
- Guard against **thundering herds, penetration, hot keys, and cold starts**, and don't cache strongly-consistent, low-reuse data.

COMING NEXT

Chapter 9 — API Gateways & the Edge. We complete the "front door" of the system: a single smart entry point that routes, authenticates, rate-limits, and aggregates — building directly on the reverse proxy of Chapter 7.